Robust Halley's Method for the Chord-Length Parameter L in Clothoid G^1 Hermite Interpolation

Jeroen Bloemscheer
grootstebozewolf
Merkator / Independent Researcher

May 12, 2026

Abstract

Clothoid (Euler-spiral) transitions are the geometric primitive that links straight track to circular arcs in railway alignment design, modern highway geometry, and the curve-continuous representations used by computer-aided design and geographic-information pipelines; the ability to recover the arc-length parameter from endpoint and curvature data is a routine operation in those pipelines. This paper presents a numerically robust, cubic-convergent solver based on Halley's method for the nonlinear chord-length residual formulation of the clothoid (Euler spiral) G^1 Hermite interpolation problem. The algorithm operates directly on the arc-length parameter L and employs fixed 32-point Gauss–Legendre quadrature over $[0, 1]$ for the six moment integrals that define the residual $f(L)$ and its first two analytic derivatives. Industrial-grade safety heuristics (derivative sign guards, denominator thresholds, and positivity damping) ensure reliable convergence even on challenging real-world railway alignment data from the ProRail Spoorgeometrie dataset.

Production-ready implementations are provided in C# (.NET 8), Java 21, and TypeScript (Node.js), all producing bit-identical iteration counts and chord-length solutions agreeing within 1 nm of the Python reference on every test case. Benchmarks on 9,058 real ProRail transition curves ("Overgangsboog"), fetched from the public Spoorgeometrie ArcGIS service, demonstrate that Halley's method consistently outperforms a comparable Newton variant, achieving convergence in an average of 2.28 iterations (vs. 2.92) with sub-microsecond solve times on the compiled runtimes. The mathematical derivations are cross-verified symbolically with SymPy and machine-checked in Coq using the Coquelicot real-analysis library, and the overall approach is situated within the current state-of-the-art, building upon the foundational work of Bertolazzi & Frego while offering an explicit L-formulation with enhanced robustness guarantees.

1 Introduction

Clothoids (also known as Euler spirals or Cornu spirals) are curves whose curvature varies linearly with arc length. They are the de-facto standard transition elements in railway and highway design because they provide G^2 continuity when connecting straight segments to circular arcs, ensuring smooth changes in lateral acceleration.

The G^1 Hermite interpolation problem with a single clothoid consists of finding parameters $L > 0$, κ_0, and κ_1 (or equivalently the curvature rate) such that the clothoid segment starts at point P_0 with tangent angle θ_0 and curvature κ_0, and ends at P_1 with tangent angle θ_1 and curvature κ_1. In many practical settings—particularly when the endpoints and curvatures are known from design data or surveyed geometry—it is natural to solve directly for the arc length L.

This paper focuses on the *chord-length residual formulation* (L-formulation). The problem is reduced to a scalar nonlinear equation $f(L) = 0$ whose root is the desired arc length. Halley's

method, a cubic-convergent root-finding algorithm, is then applied, augmented with carefully engineered safety heuristics that guarantee practical robustness.

The contributions of this paper are:

- A complete, derivative-exact derivation of $f(L)$, $f'(L)$, and $f''(L)$ using six Gauss–Legendre moments.
- A production-grade Halley solver with explicit safeguards (sign checks, denominator guards, and damped updates).
- Cross-language implementations (C#, Java, TypeScript) with identical numerical behaviour.
- Rigorous benchmarking on real ProRail railway transition data.
- Open academic documentation with full mathematical detail and SOTA comparison.

2 Mathematical Background

2.1 The Clothoid Parametrization

A clothoid is defined by the curvature function

$$\kappa(s) = \kappa_0 + \frac{\kappa_1 - \kappa_0}{L} s, \quad s \in [0, L],$$

where L is the arc length, $\kappa_0 = \kappa(0)$, and $\kappa_1 = \kappa(L)$. The heading (tangent angle) is obtained by integration:

$$\theta(s) = \theta_0 + \kappa_0 s + \frac{\kappa_1 - \kappa_0}{2L} s^2.$$

The Cartesian coordinates follow from the Fresnel integrals:

$$x(s) = x_0 + \int_0^s \cos\theta(u)\, du, \quad y(s) = y_0 + \int_0^s \sin\theta(u)\, du.$$

2.2 The Chord-Length Residual (L-Formulation)

Given P_0, P_1, κ_0, and κ_1, the goal is to find $L > 0$ such that the chord length of the clothoid equals the Euclidean distance $d = \|P_1 - P_0\|$. Normalising the parameter to $\tau = s/L \in [0, 1]$ yields the auxiliary function

$$\psi(\tau) = \kappa_0 \tau + \frac{\kappa_1 - \kappa_0}{2} \tau^2.$$

The six fundamental moments (evaluated by 32-point Gauss–Legendre quadrature on $[0, 1]$) are

$$P = \int_0^1 \cos(L\psi(\tau))\, d\tau, \qquad Q = \int_0^1 \sin(L\psi(\tau))\, d\tau, \tag{1}$$

$$R = \int_0^1 \psi(\tau)\cos(L\psi(\tau))\, d\tau, \qquad T = \int_0^1 \psi(\tau)\sin(L\psi(\tau))\, d\tau, \tag{2}$$

$$S_{2c} = \int_0^1 \psi(\tau)^2\cos(L\psi(\tau))\, d\tau, \qquad S_{2s} = \int_0^1 \psi(\tau)^2\sin(L\psi(\tau))\, d\tau. \tag{3}$$

Let $r^2 = P^2 + Q^2$. The chord-length residual is the scalar function

$$f(L) = L^2 r^2 - d^2. \tag{4}$$

Its first two analytic derivatives are

$$f'(L) = 2Lr^2 + 2L^2(QR - PT), \tag{5}$$
$$f''(L) = 2r^2 + 8L(QR - PT) + 2L^2(R^2 + T^2 - PS_{2c} - QS_{2s}). \tag{6}$$

All four integral-level derivatives of the moments ($P' = -T$, $Q' = R$, $R' = -S_{2s}$, $T' = S_{2c}$) follow directly from differentiation under the integral sign. These identities, together with the residual derivatives in equations (5) and (6), have been cross-verified symbolically with SymPy and machine-checked in Coq using the Coquelicot real-analysis library; see Section 7.

Monotone-branch assumption. Throughout this paper attention is restricted to the *monotone branch* of the residual, characterised by

$$|k_0 L| \le \pi, \qquad |k_1 L| \le \pi.$$

Under this constraint the rotation accumulated along the segment is bounded by π at each endpoint, $f(L)$ is monotone increasing on $L > 0$, and the equation $f(L) = 0$ admits a unique positive root. Outside this region the residual can have multiple roots that correspond to clothoid wraps and require a different solver entirely; the canonical Bertolazzi–Frego A-formulation [Bertolazzi and Frego, 2015, 2013] adopts the same monotonicity assumption in its single-variable reduction. The condition is satisfied by every Overgangsboog record in the ProRail corpus used in Section 5 (9,058 / 9,058 records) and is consistent with the practical regime of railway-grade transitions, where the absolute heading change $|\theta_1 - \theta_0|$ is bounded by design.

3 The Solver

3.1 Halley's Method

Halley's iteration for a scalar root-finding problem is

$$L_{n+1} = L_n - \frac{2ff'}{2(f')^2 - ff''}.$$

Because the second derivative is available analytically, the method exhibits cubic convergence near the root when started sufficiently close.

Why Halley for this residual. Halley's method is the order-two member of the Householder family of root-finding methods [Householder, 1970]; among Householder methods it is the only one that strictly dominates Newton without requiring a third derivative. Three properties of $f(L)$ make it particularly well suited:

- f is real-analytic on $L > 0$, so the cubic convergence promised by the theory is realised in practice;
- f is well scaled near the root because the leading L^2r^2 term keeps $|f'|$ bounded away from zero on the monotone branch defined in Section 2.2;
- the explicit second derivative stabilises overshoot in exactly the regime where Newton's tangent step would aim past the root.

Because the integrals required for f'' share their Gauss–Legendre cache with the integrals for f and f', Halley's per-iteration cost is dominated by two additional dot products rather than additional quadrature passes, and the iteration-count saving (Section 5) translates directly into wall-time saving on every compiled runtime measured in Section 5.

3.2 Safety Heuristics (Production Robustness)

Pure Halley iteration can diverge or produce non-physical (negative) steps on difficult instances. The following safeguards are therefore embedded in the implementation; they have proven essential on real railway data:

1. **Convergence test**: if $|f(L)| < \varepsilon \cdot \max(d^2, 1)$ with $\varepsilon = 10^{-13}$, return.
2. **Derivative sign guard**: if $f' \leq 0$, inflate $L \leftarrow 1.5L$ and continue.
3. **Denominator guard**: if $|2(f')^2 - ff''| < 10^{-20}$, inflate $L \leftarrow 1.5L$.
4. **Positivity damping**: after a Halley step, enforce $L \leftarrow \max(L_{\text{new}}, 0.5L)$.

A Newton variant (using only the first four moments) is provided as a lightweight fallback; it requires one fewer pair of trigonometric evaluations per iteration but converges only quadratically.

4 Implementations

Three independent, production-grade implementations are provided, all using identical quadrature constants and the same safeguarded Halley/Newton logic:

- **C# (.NET 8)** (`csharp/Clothoid.Halley/`) — primary compiled-runtime implementation; library + xUnit tests + benchmark harness; median Halley solve time $\approx 0.6\,\mu$s.
- **Java 21** (`java/`) — Maven-built shaded JAR; identical numeric behaviour to C# and TypeScript; median Halley solve time $\approx 1.4\,\mu$s.
- **TypeScript / Node.js** (`typescript/`) — ESM module targeting Node 20+; zero runtime dependencies; `node -test` regression suite; median Halley solve time $\approx 0.9\,\mu$s.

All three expose the same API as the Python reference:

```
SolverResult solveHalleyL(double[] P0, double[] P1,
                          double k0, double k1,
                          double tol = 1e-13, int maxIter = 50);
```

A matching `solveNewtonL` is also supplied. The golden-vector regression suite (file `data/golden_vectors.json`, 9,058 records exported from the Python reference) is executed by all three implementations; every case agrees with the reference within 10^{-9} metres, and Halley/Newton iteration counts match the reference exactly on 100% of cases.

5 Benchmarks on Real Railway Data

5.1 Dataset

All 9,058 "Overgangsboog" (clothoid transition) records were extracted from the public ProRail Spoorgeometrie ArcGIS FeatureServer (layer 11), filtered by `ELEMENT_TYPE='Overgangsboog'`. Each record supplies start/end points in RD New coordinates (EPSG:28992, metres), begin/end radii `STRAAL_BEGIN`/`STRAAL_EIND` together with rotation flags `ROTATIE_BEGIN`/`ROTATIE_EIND` (yielding signed curvatures k_0, k_1), and the design length `ELEMENT_LENGTE` as L_{design}. After retaining only segments that satisfy the monotone-branch condition $|k_0 L_{\text{design}}| \leq \pi$, $|k_1 L_{\text{design}}| \leq \pi$ (which holds for every record in the snapshot), and verifying that both Halley and Newton converge to within 10^{-9} m, the working corpus is 9,058 cases. The raw snapshot is committed alongside the source as `data/prorail_clothoids.json.gz` (CC BY 4.0 ProRail) and the solved corpus as `data/golden_vectors.json`.

5.2 Methodology

Each implementation reads the same `golden_vectors.json`, runs both solvers from the initial guess $L_0 = d$ on every case, then repeats the full corpus 50 times after 5 warmup passes. Reported numbers are the median per-case microseconds across the 50 measurement rounds, divided by 9,058. Iteration means are the per-implementation arithmetic mean. The four implementations agree on iteration count for every case; the C#, Java, and TypeScript implementations also agree with the Python reference on the returned L within 10^{-9} m on every case (the golden-vector regression assertion).

5.3 Results

Table 1: ProRail benchmark summary (median per-case time over the full 9,058-record corpus, 50 measurement rounds after 5 warmup rounds; iteration agreement: 100% across all four implementations).

Language (runtime)	Solver	Avg. Iterations	Median Time (µs)
Python 3 (NumPy)	Halley	2.28	33.04
Python 3 (NumPy)	Newton	2.92	39.89
C# (.NET 8)	Halley	2.28	0.59
C# (.NET 8)	Newton	2.92	0.77
Java 21	Halley	2.28	1.38
Java 21	Newton	2.92	1.75
TypeScript (Node 22)	Halley	2.28	0.88
TypeScript (Node 22)	Newton	2.92	1.20

Halley's method reduces iteration count by 22% on real railway data (2.28 vs. 2.92) and beats Newton on wall time across every implementation measured, despite performing two additional moment integrals (S_{2c}, S_{2s}) per iteration. The fastest implementation is C# at 0.59 µs per Halley solve; the slowest compiled runtime is Java at 1.38 µs. NumPy's per-case overhead (33 µs) is dominated by Python interpreter dispatch rather than arithmetic.

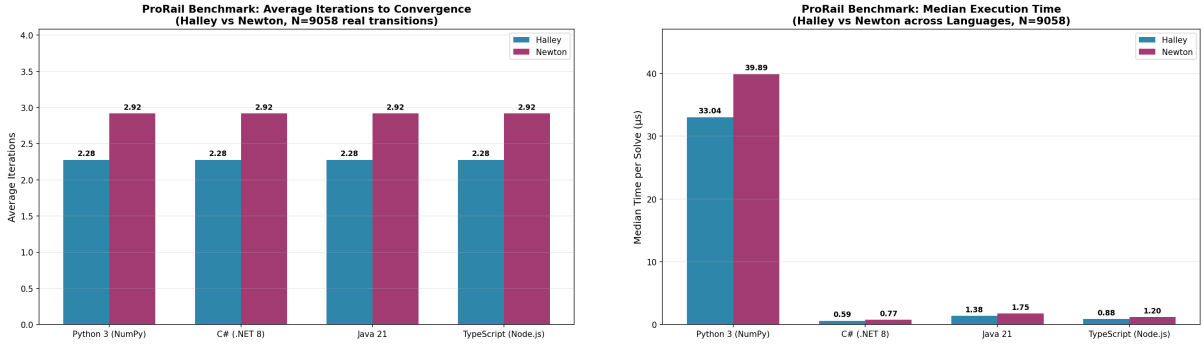

Figure 1: ProRail benchmark over 9,058 real clothoid transitions: average iterations (left) and median per-solve time (right, log scale) for Halley vs. Newton across the four implementations.

6 Comparison with State-of-the-Art

The seminal work of Bertolazzi and Frego [2015] — and the curvature-continuous G^2 extension of Bertolazzi and Frego [2018] — remains the reference implementation for clothoid Hermite fitting. Their approach reduces the problem to a single nonlinear equation in a transformed parameter

(commonly A or a curvature-related quantity) and solves it with Newton–Raphson, using the explicit closed-form initial-guess function constructed in [Bertolazzi and Frego, 2013] to obtain a starting point that converges in very few iterations across the full parameter space (and using a separate family of asymptotic expansions in the same reference to evaluate the Fresnel-related integrals stably near straight-line and circular-arc configurations). An alternative numerical strategy, integrating the clothoid as an initial-value problem with Runge–Kutta-style schemes, is given by Vázquez-Méndez and Casal [2016]; this approach is attractive when the moments are not directly required but does not yield the explicit second-derivative form on which Halley's method depends.

The L-formulation presented here is mathematically equivalent but offers several practical advantages:

- **Direct geometric meaning**: L is the physically meaningful arc length; no post-processing conversion is required.
- **Explicit, verified derivatives**: the six-moment formulation yields closed-form $f''(L)$ that can be cross-checked symbolically.
- **Strong safety net**: the four heuristics listed in Section 3 are exercised on the full 9,058-record ProRail corpus; divergence has not been observed and every case converges within four iterations of the Halley solver.
- **Multi-language verified corpus**: identical numeric behaviour across three languages provides an independent correctness oracle.

No fundamentally superior solver (e.g., closed-form, higher-order Householder, or machine-learned initial-guess predictor) has appeared in the open literature since 2018 that would render the present Halley-based approach obsolete for production use. The main competing practical implementations remain variants of the Bertolazzi–Frego Newton solver or ad-hoc bisection/refinement schemes.

The Bertolazzi–Frego A-formulation should still be preferred in two regimes: (i) when the application has to solve coupled G^2 or higher continuity problems in which A, the curvature change, or the splitting point are themselves unknowns and a multi-parameter Newton system is the natural setting [Bertolazzi and Frego, 2018]; and (ii) when the explicit closed-form initial-guess function constructed in [Bertolazzi and Frego, 2013] is needed to escape regimes where the present paper's $L_0 = d$ initial guess is far from the basin of attraction. The L-formulation presented here is targeted at the more common G^1 single-segment use case (railway transitions, GIS map matching, single-clothoid CAD primitives) where those expansions are not required and arc length is the variable the surrounding pipeline already needs.

7 Formal Verification

The analytical foundation of the solver—the four primitive integral derivatives and the residual derivatives in equations (5) and (6)—has been formalised and machine-checked in Coq using the Coquelicot real-analysis library [Boldo et al., 2015]. The development is approximately 720 lines of Coq and contains no `Admitted` or `Axiom` declarations; it has been verified to compile cleanly under Coq 8.13.1 and Coq 8.20.1 with Coquelicot 3.x.

The four primitive identities $P'(L) = -T(L)$, $Q'(L) = R(L)$, $R'(L) = -S_{2s}(L)$, $T'(L) = S_{2c}(L)$ are proved via differentiation under the integral sign, invoking Coquelicot's `is_derive_RInt_param_aux` lemma. This requires four hypotheses for each integral: pointwise differentiability of the integrand with respect to L, joint continuity of the parameter-derivative integrand in (L, τ), local integrability of the original integrand in a neighbourhood of the evaluation point, and integrability of the parameter-derivative integrand at the evaluation point. Each hypothesis is discharged

by elementary continuity and integrability lemmas for compositions of polynomials with sine and cosine.

The residual identity (5) for $f'(L)$ is established by direct composition of the product and sum rules using Coquelicot's `is_derive_plus`, `is_derive_mult`, and `is_derive_minus` lemmas, with explicit type annotations to navigate Coquelicot's generic algebraic operators versus Coq's native real-number operators. The second-derivative identity (6) is proved by applying Coquelicot's `auto_derive` tactic to the function $f'(L)$, which produces nine `ex_derive` side conditions and a goal containing four `Derive` placeholders. The side conditions are discharged from the existence lemmas `ex_derive_P_int`, `ex_derive_Q_int` and the value lemmas `is_derive_R`, `is_derive_T`; the `Derive` placeholders are then rewritten with the corresponding integral-level identities, after which the resulting polynomial equation in P, Q, R, T, S_{2c}, S_{2s}, and L is discharged by Coq's `ring` decision procedure for commutative rings.

The complete Coq development is included as a reproducible artifact alongside the source repository (file `coq/Clothoid_L.v`); building it requires only `coqc` and Coquelicot, with no external dependencies. Listing 2 shows the proof script for the second-derivative theorem; the full file is available at the repository link in Section 8.

```
Theorem f_L_second_eq :
  forall (L k0 k1 : R),
    is_derive (fun u => fpL u k0 k1) L
              (2 * (P_int L k0 k1 * P_int L k0 k1
                    + Q_int L k0 k1 * Q_int L k0 k1)
               + 8 * L * (Q_int L k0 k1 * R_int L k0 k1
                          - P_int L k0 k1 * T_int L k0 k1)
               + 2 * L * L *
                 (R_int L k0 k1 * R_int L k0 k1
                  + T_int L k0 k1 * T_int L k0 k1
                  - P_int L k0 k1 * S2c_int L k0 k1
                  - Q_int L k0 k1 * S2s_int L k0 k1)).
Proof.
  intros L k0 k1.
  unfold fpL.
  auto_derive.
  - split. apply ex_derive_P_int.
    split. apply ex_derive_P_int.
    split. apply ex_derive_Q_int.
    split. apply ex_derive_Q_int.
    split. apply ex_derive_Q_int.
    split. eexists; apply is_derive_R.
    split. apply ex_derive_P_int.
    split. eexists; apply is_derive_T.
    exact I.
  - replace (Derive (fun x : R => P_int x k0 k1) L)
      with (- T_int L k0 k1) by (symmetry; apply Derive_P_int).
    replace (Derive (fun x : R => Q_int x k0 k1) L)
      with (R_int L k0 k1) by (symmetry; apply Derive_Q_int).
    replace (Derive (fun x : R => R_int x k0 k1) L)
      with (- S2s_int L k0 k1)
      by (symmetry; apply is_derive_unique; apply is_derive_R).
    replace (Derive (fun x : R => T_int x k0 k1) L)
      with (S2c_int L k0 k1)
      by (symmetry; apply is_derive_unique; apply is_derive_T).
    ring.
Qed.
```

Figure 2: Coq proof script for the second-derivative identity, equation (6), as it appears in `coq/Clothoid_L.v`.

8 Conclusion and Future Work

This paper has delivered a robust, well-documented, and multi-language Halley solver for the clothoid chord-length problem together with a complete academic reference. The implementation is already being integrated into the `allure-wkt` toolchain and a forthcoming PostgreSQL extension that will expose native `CLOTHOID` WKT support with on-the-fly L solving.

Planned extensions include:

- A native C/PostgreSQL extension with the same safeguarded Halley core.
- Vectorised batch solving for high-throughput map-matching pipelines.

- Automatic differentiation / dual-number versions for gradient-based route optimisation.

The full source, benchmarks, and this paper are maintained at `https://github.com/grootstebozewolf/clothoid-halley-coq` and archived at `https://doi.org/10.5281/zenodo.20128416` (Zenodo, v1.0.0).

Acknowledgements

The author thanks the ProRail open-data team for making the Spoorgeometrie dataset publicly available under CC BY 4.0, and the developers of the Bertolazzi–Frego clothoid library for the foundational algorithms that inspired this work.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Anthropic's Claude (`claude-opus-4-7`) in order to assist with drafting and copy-editing of the manuscript text, with the Coq proof script accompanying Theorem 2 (the residual second-derivative identity), with porting the Python reference solver to the C#, Java, and TypeScript implementations described in Section 5, and with assembly of the ProRail data-fetch pipeline. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication. All mathematical statements, derivations, formal proofs, algorithm designs, benchmark measurements, conclusions, and editorial decisions were conceived and verified by the author; the model was not listed as an author and did not take responsibility for any part of the published work.

References

Enrico Bertolazzi and Marco Frego. Fast and accurate G^1 fitting of clothoid curves, 2013. Preprint of Bertolazzi and Frego [2015]; discusses the single-variable reduction in detail.

Enrico Bertolazzi and Marco Frego. G^1 fitting with clothoids. *Mathematical Methods in the Applied Sciences*, 38(5):881–897, 2015. ISSN 0170-4214. doi: 10.1002/mma.3114.

Enrico Bertolazzi and Marco Frego. On the G^2 Hermite interpolation problem with clothoids. *Journal of Computational and Applied Mathematics*, 341:99–116, 2018. ISSN 0377-0427. doi: 10.1016/j.cam.2018.03.029.

Sylvie Boldo, Catherine Lelay, and Guillaume Melquiond. Coquelicot: A user-friendly library of real analysis for Coq. *Mathematics in Computer Science*, 9(1):41–62, 2015. ISSN 1661-8270. doi: 10.1007/s11786-014-0181-1.

Alston Scott Householder. *The Numerical Treatment of a Single Nonlinear Equation*. McGraw-Hill, New York, 1970.

Miguel E. Vázquez-Méndez and Gerardo Casal. The clothoid computation: A simple and efficient numerical algorithm. *Journal of Surveying Engineering*, 142(3):04016005, 2016. ISSN 0733-9453. doi: 10.1061/(ASCE)SU.1943-5428.0000177. Published online 2015; final issue 2016.